

COMMUNITY WATCH: ACADEMIC PROJECT REPORT

Project Title: Community Watch: A Real-Time Community Insecurity Reporting and Incident Monitoring Mobile System

Discipline: Computer Science / Software Engineering | **Classification:** Final-Year / Capstone Project

1. Background of the Study

In contemporary urban and semi-urban communities, safety and security remain fundamental prerequisites for socio-economic development and quality of life. However, maintaining effective community surveillance and public security has grown increasingly challenging due to rapid urbanization, expanding residential perimeters, and strained law enforcement resources.

Traditionally, community insecurity reporting has relied heavily on archaic and fragmented communication channels, including:

- In-person visits to local police precincts,
- Voice telephone calls to emergency hotlines (which frequently suffer from line congestion, long hold times, or lack of coverage),
- Unverified word-of-mouth alerts, and
- Disorganized, unmoderated social media threads or messaging groups (e.g., WhatsApp, Telegram).

While social media and messaging groups have demonstrated citizen eagerness to share safety alerts, they suffer from significant operational drawbacks:

1. **Lack of Structure:** Incident messages lack standardized classification (e.g., distinguishing between vandalism, armed robbery, fire outbreaks, or suspicious activities).
2. **Missing Spatial Data:** Descriptions such as "*near the junction with the yellow kiosk*" fail to provide actionable, machine-readable GPS coordinates necessary for emergency dispatches.
3. **No Accountable Feedback Loop:** Traditional reports vanish into bureaucratic voids; reporting citizens rarely receive status updates or verification on whether responding units have addressed the incident.

With the global proliferation of smartphones equipped with global positioning systems (GPS), high-resolution cameras, and ubiquitous mobile broadband, mobile computing offers a transformative opportunity for community policing and crowd-sourced intelligence.

Community Watch was conceived as an intelligent, secure, and user-centric mobile solution that bridges the communication divide between community residents and security responders. Leveraging the **Flutter** framework for cross-platform client responsiveness, **OpenStreetMap** for geographic spatial visualization, and **Supabase** (PostgreSQL with Row-Level Security) for cloud database and media persistence, the system provides a unified ecosystem for crowdsourced incident reporting, real-time status tracking, interactive hazard mapping, emergency distress dialing, and incident-level community collaboration.

2. Problem Statement

Despite advancements in consumer telecommunications, community safety management continues to face acute systemic challenges:

1. **Delayed Incident Reporting and Slow Emergency Response:** Traditional reporting mechanisms require substantial human mediation before responders can be mobilized. In life-threatening scenarios (such as ongoing robberies, fires, or physical assaults), every minute spent locating an emergency contact number or describing landmark directions severely degrades emergency outcomes.
2. **Inaccurate Geographical and Verifiable Incident Data:** Textual and verbal descriptions of incident locations are prone to subjectivity, error, and ambiguity. Furthermore, responders frequently lack preliminary photographic evidence before arriving at a scene, impairing their ability to assess threat severity or allocate appropriate personnel and equipment.
3. **Absence of Transparency and Citizen Engagement:** In conventional reporting paradigms, citizens possess zero visibility into the lifecycle of their reported cases. This lack of transparency fosters apathy and skepticism among community members, discouraging civic participation in neighborhood crime prevention.
4. **Information Fragmentation and Disinformation:** Unstructured alerts circulated across casual messaging channels often generate mass panic, propagate unverified rumors, and lack official responder moderation or status verifications.
5. **Resource and Infrastructure Barriers for Local Neighborhoods:** Enterprise municipal dispatch software solutions are prohibitively expensive and complex for local residential committees, university campuses, and estate associations. There is an urgent necessity for an accessible, low-latency, cross-platform mobile solution that democratizes community-level incident management.

3. Aim of the Project

*To design, develop, and deploy a responsive, cross-platform mobile application and cloud-backed monitoring system—**Community Watch**—that enables citizens and residential communities to report insecurity incidents in real time with high-precision GPS positioning and visual evidence,*

track incident resolution lifecycles, view community hazard maps, access emergency hotlines, and engage in moderated situational updates.

4. Objectives of the Study

To fulfill the primary aim, the study addresses the following specific technical and research objectives:

1. System Requirements and Architectural Modeling:

- Investigate and analyze user, community, and administrative requirements for community incident monitoring.
- Design a secure multi-tier system architecture connecting client mobile devices with an asynchronous cloud backend.

2. Secure User Authentication and Role-Based Access Control:

- Implement cryptographically secure user authentication (sign-up, sign-in, session recovery) via Supabase Auth.
- Enforce database-level **Row-Level Security (RLS)** policies that demarcate privileges between standard community residents and elevated system administrators.

3. Geolocated Incident Reporting Engine:

- Develop an intuitive incident submission interface enabling citizens to categorize incidents (e.g., *Theft, Robbery, Assault, Vandalism, Fire, Suspicious Activity*).
- Integrate device GPS hardware using `geolocator` to capture high-accuracy latitude and longitude coordinates.
- Implement automated reverse-geocoding via `geocoding` to translate raw GPS coordinates into recognizable street addresses.
- Enable media attachment through camera/gallery image capture and cloud storage bucket persistence.

4. Interactive Spatial Mapping and Visualization:

- Integrate an open-source mapping engine using `flutter_map` and Leaflet-based OpenStreetMap tiles.
- Plot active community incident markers dynamically on the map, color-coded according to investigation status (*Pending, Under Investigation, Resolved, Rejected*).
- Provide interactive marker touch inspection displaying summary preview cards with deep linking to full incident dossiers.

5. Emergency SOS Rapid-Dialing Interface:

- Engineer an immediate, high-priority **Emergency SOS** quick-action module on the home dashboard.

- Integrate native telephony protocols via `url_launcher` allowing one-tap speed dialing to verified law enforcement, medical ambulance, fire rescue, and neighborhood patrol dispatchers.

6. Incident Lifecycle Management & Administrative Dashboard:

- Construct a dedicated administrative monitoring dashboard featuring animated real-time statistical metrics (Total Reports, Pending, Investigating, Resolved).
- Provide administrative tooling to review incoming incident dossiers and transition case statuses across their lifecycle.

7. Collaborative Discussion and Situational Updates:

- Engineer a multi-user, real-time discussion thread beneath individual incident reports.
- Implement automated verified badging (`[OFFICIAL]`) to distinguish official responder advisories from civilian witness commentary.

8. Dynamic Multi-Criteria Filtering:

- Develop real-time filtering mechanisms across both the structured list view and the spatial map view based on incident category and investigation status.

5. Scope and Delimitations of the Study

In-Scope:

- **Client Application:** Cross-platform mobile client built using Dart and Flutter, targeting Android (and iOS-ready architecture).
- **Backend Infrastructure:** Managed PostgreSQL relational database with automated schema migrations, storage buckets, and serverless authentication provided by Supabase.
- **Geographic Services:** GPS positioning, reverse address lookup, and interactive tile rendering without proprietary API billing dependencies.
- **Telephony Integration:** Direct invocation of device dialers for emergency hotlines.
- **Security:** Strict Row-Level Security (RLS) ensuring users manage only their own submissions while preserving public read transparency for community safety.

Delimitations (Out-of-Scope):

- Automated hardware-based alarm integration (e.g., physical sirens or CCTV feeds).
- SMS cellular fallback during periods of total mobile internet disconnection.

6. Significance of the Study

- 1. **For Citizens & Residents:** Provides an accessible, empowering tool to document threats without fear, seek emergency assistance in seconds, and stay informed about safety developments in their immediate surroundings.
- 2. **For Law Enforcement & Community Patrols:** Delivers structured, geocoded, and photo-verified intelligence, drastically improving response times, route planning, and resource allocation.
- 3. **For Community Leadership & Estate Associations:** Offers actionable data analytics on crime patterns, incident hotspots, and resolution efficiency, supporting data-driven security policies.
- 4. **Academic & Technical Contribution:** Demonstrates how modern cross-platform toolchains (Flutter), serverless SQL architectures (Supabase with RLS), and open-source spatial cartography (OpenStreetMap) can be synergized to build high-performance, cost-effective public safety solutions without dependence on proprietary cloud mapping licenses.

7. System Architecture Overview

Component Layer	Technology Employed	Core Responsibilities
Presentation Tier	Flutter (Dart 3.x), Material 3, Motion System	Mobile UI, micro-animations, form validation, dynamic filtering, map rendering
Spatial / Geolocation	<code>geolocator</code> , <code>geocoding</code> , <code>flutter_map</code> , <code>latlong2</code>	GPS acquisition, reverse geocoding, OpenStreetMap tile cache, marker positioning
Hardware Integration	<code>image_picker</code> , <code>url_launcher</code>	Camera/gallery evidence capture, native telephony dialing (<code>tel:</code>)
Backend & Persistence	Supabase (PostgreSQL 15+)	Relational tables (<code>profiles</code> , <code>reports</code> , <code>report_comments</code>), media storage
Security Tier	PostgreSQL Row-Level Security (RLS)	Declarative access controls, user data isolation, admin privilege verification
DevOps / CI/CD	GitHub Actions, Git	Automated cloud Android APK and iOS IPA packaging